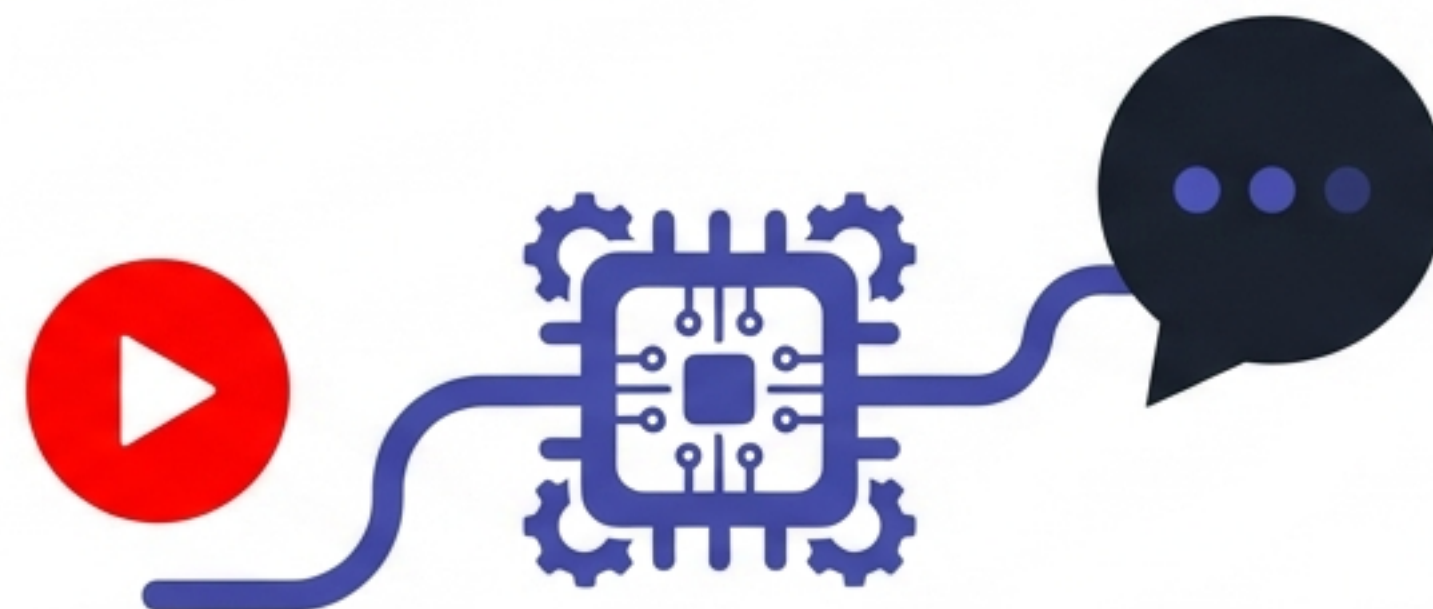


Building a YouTube Chatbot with LangChain & RAG

A practical guide to transforming video content into an interactive Q&A interface.



Based on the technical walkthrough by CampusX

The Challenge of Long-Form Video Content

Deep Indigo Inter Tight

Context

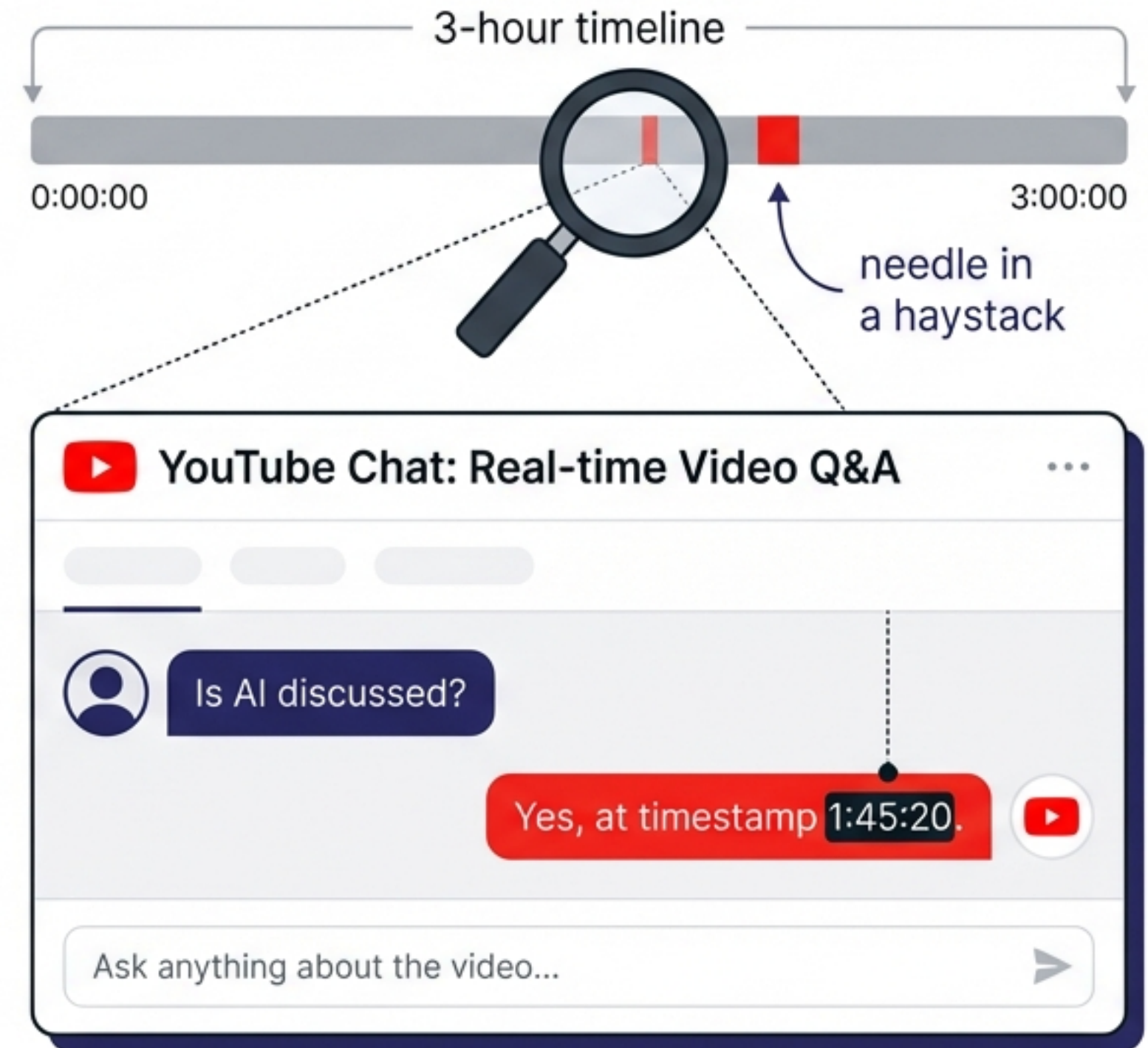
We consume hours of dense video content—podcasts, lectures, and tutorials.

The Problem

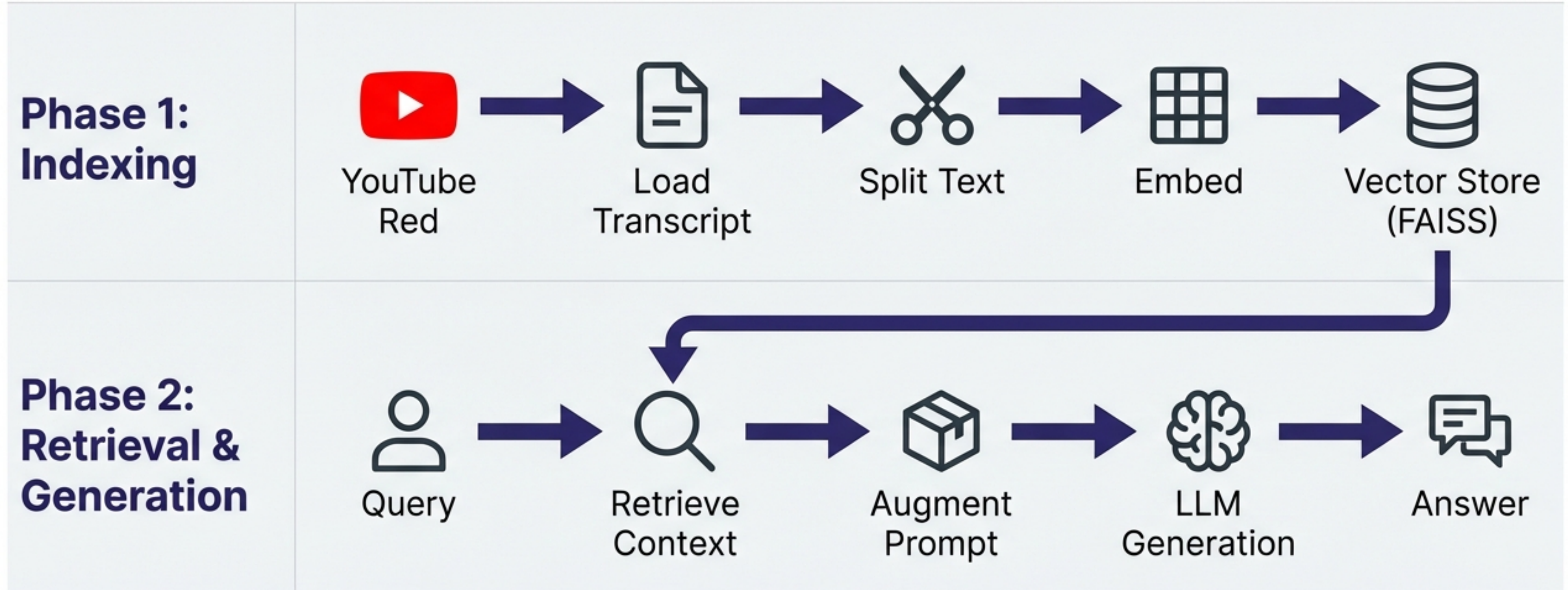
Finding specific answers in a 3-hour podcast requires watching the entire video. There is no “Ctrl+F” for video context.

The Solution

YouTube Chat: A RAG-based system allowing real-time Q&A with video transcripts.



The Blueprint: RAG Architecture



This workflow transforms unstructured video data into a queryable knowledge base.

Phase 1: Ingestion (Loading Transcripts)

- **Task**

Extracting raw text from Video ID.

- **Tool Choice**

YouTubeTranscriptApi

- **Why Not Standard Loaders?**

Direct API is more reliable than standard LangChain loaders for specific video IDs.

- **Key Detail**

Handles language selection (e.g., Hindi vs English) and requires Video ID, not full URL.

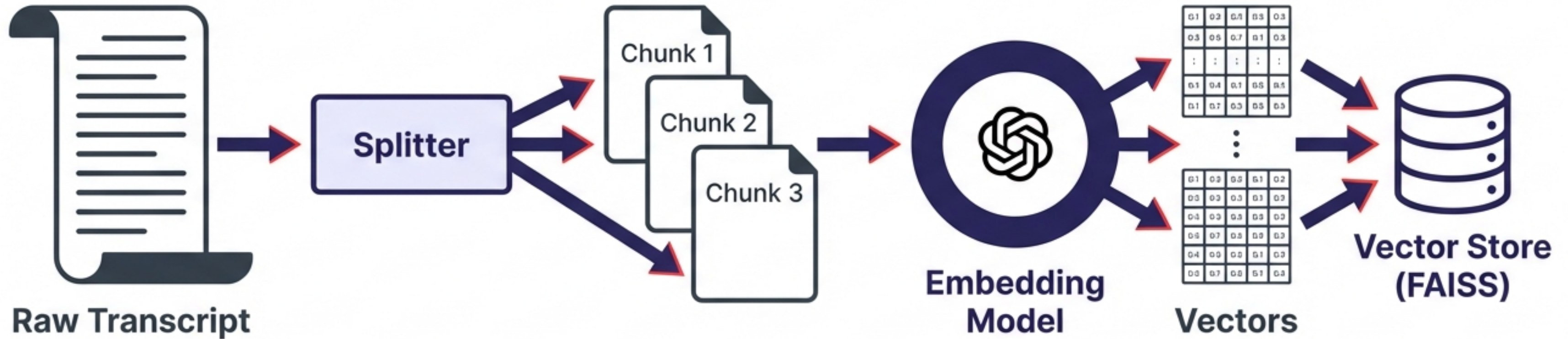
```
from youtube_transcript_api import YouTubeTranscriptApi

# Use Video ID, not full URL
video_id = "n2y-0D2n0mU"

# Fetch transcript specifying languages
transcript = YouTubeTranscriptApi.get_transcript(
    video_id,
    languages=['en', 'hi']
)

# Output: List of timestamped dictionaries
```


Phase 1: Processing (Splitting & Embedding)



Step A: Text Splitting

- **Tool:** RecursiveCharacterTextSplitter
- **Specs:** Chunk Size: 1000 | Overlap: 200
- **Logic:** Breaks transcript into manageable segments while preserving context.

Step B: Vector Storage

- **Tool:** OpenAI Embeddings + FAISS
- **Process:** Converts text chunks into 168+ searchable vectors (for a 2hr video).

Phase 2: The Retrieval Engine

Mechanism

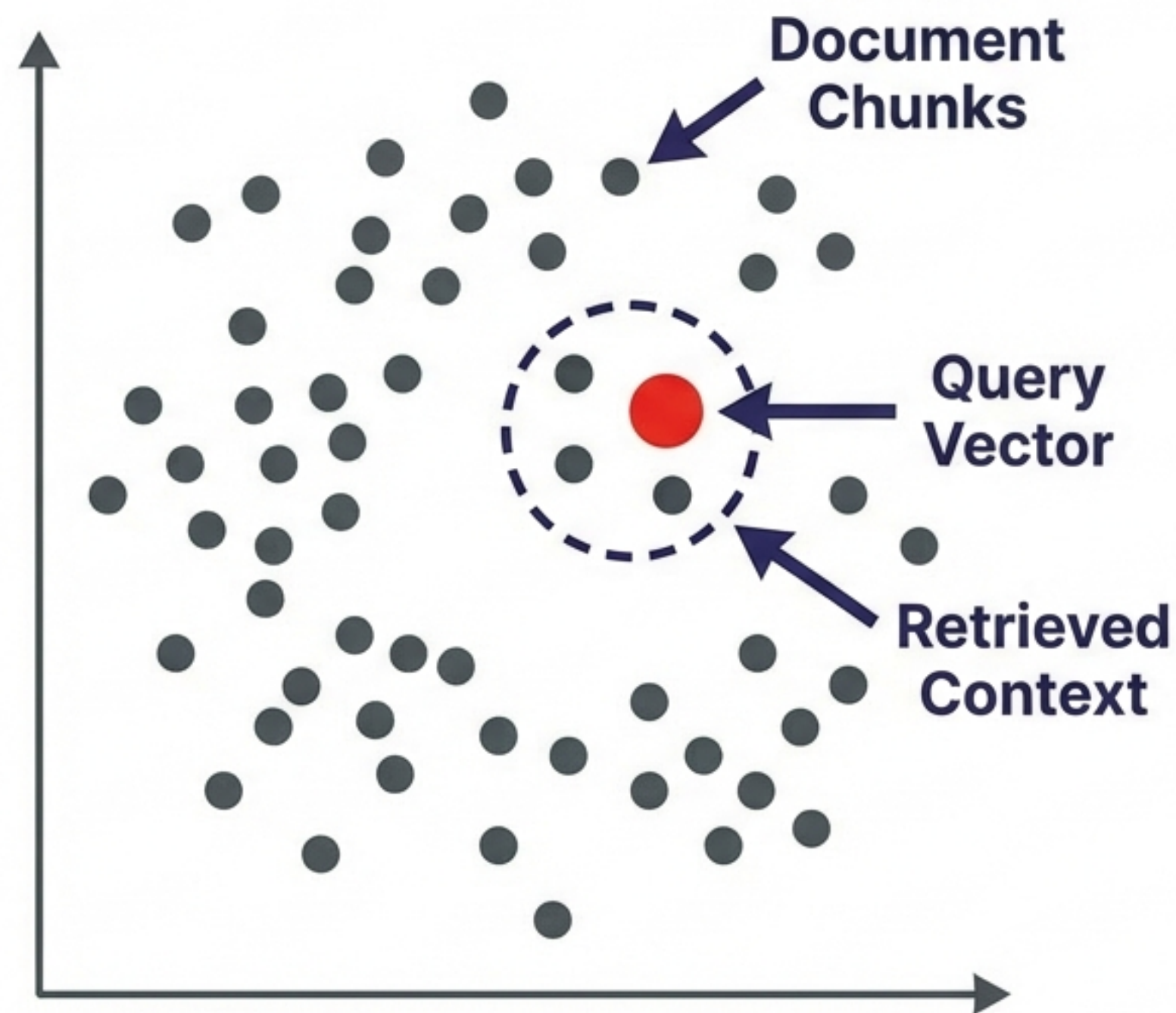
Semantic Similarity Search

The Process

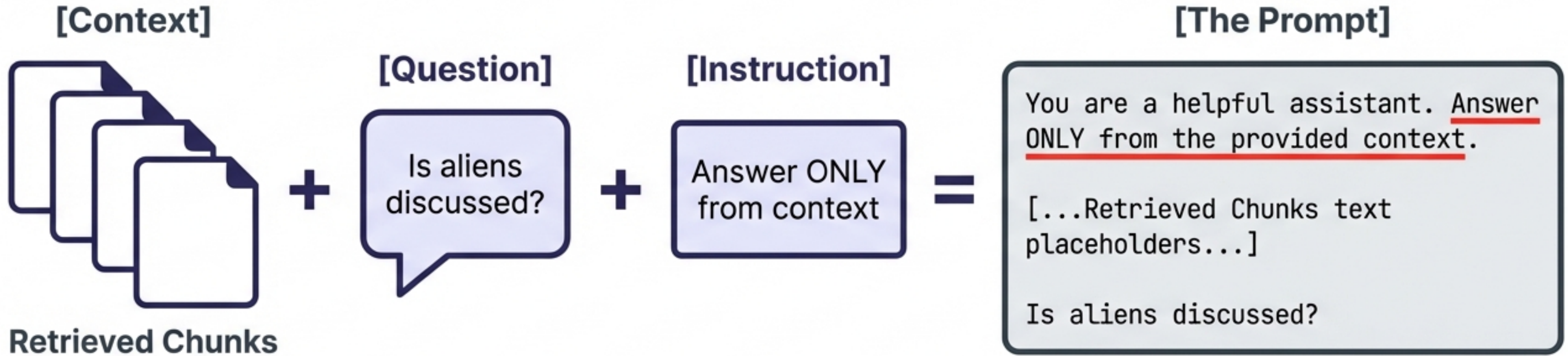
1. User asks: "What is DeepMind?"
2. Query is converted to a vector.
3. FAISS finds the closest vector matches.

Code Logic

```
retriever = vectorstore.as_retriever(  
    search_kwargs={"k": 4})
```



Phase 2: Augmentation & Generation



The Prompt Template

Inter (JetBrains Mono)

You are a helpful assistant.
Answer ONLY from the
provided context.

The Outcome

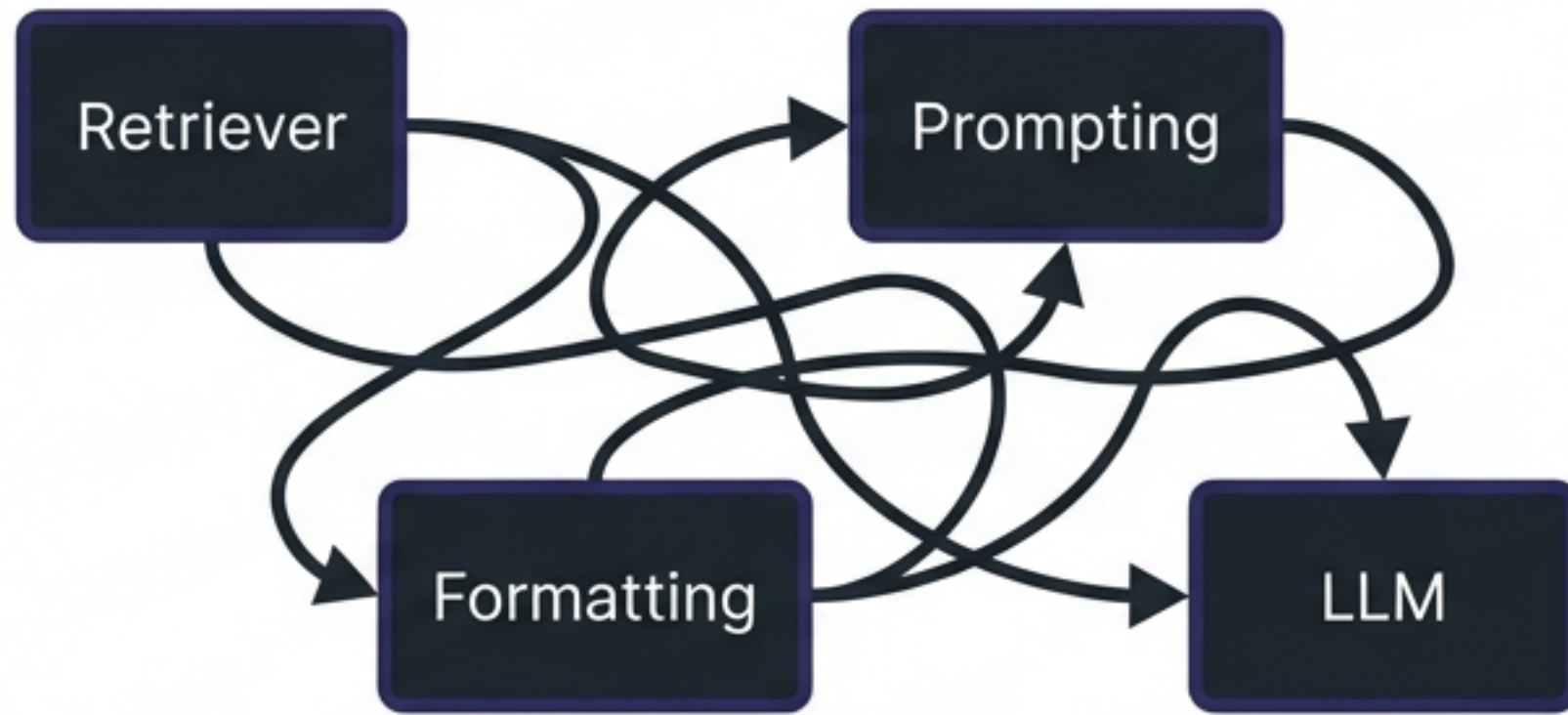
The LLM generates a
The LLM generates a coherent
answer based strictly on the
video content, reducing
hallucinations.

Status

Functional, but
Functional, but currently
relying on manual function
calls.

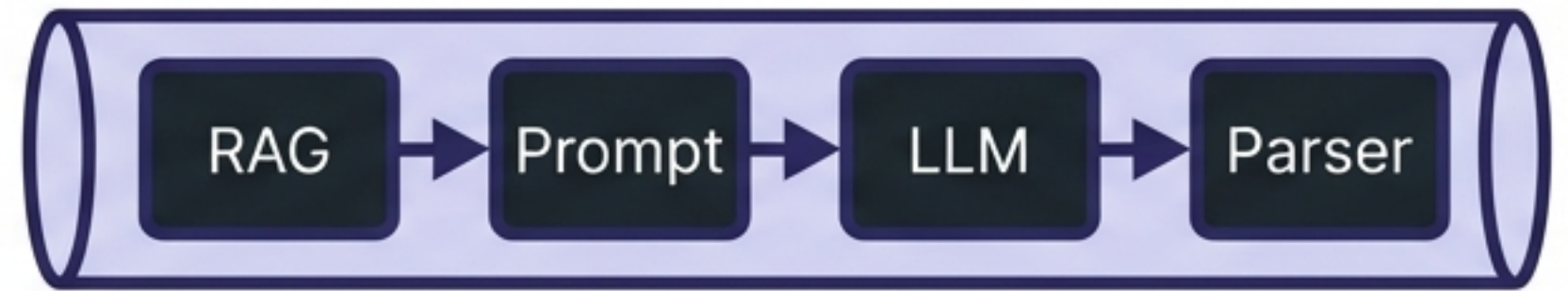
From Spaghetti Code to LCEL Chains

Manual Sequencing



Manually invoking retriever, then formatting, then prompting, then LLM is inefficient.

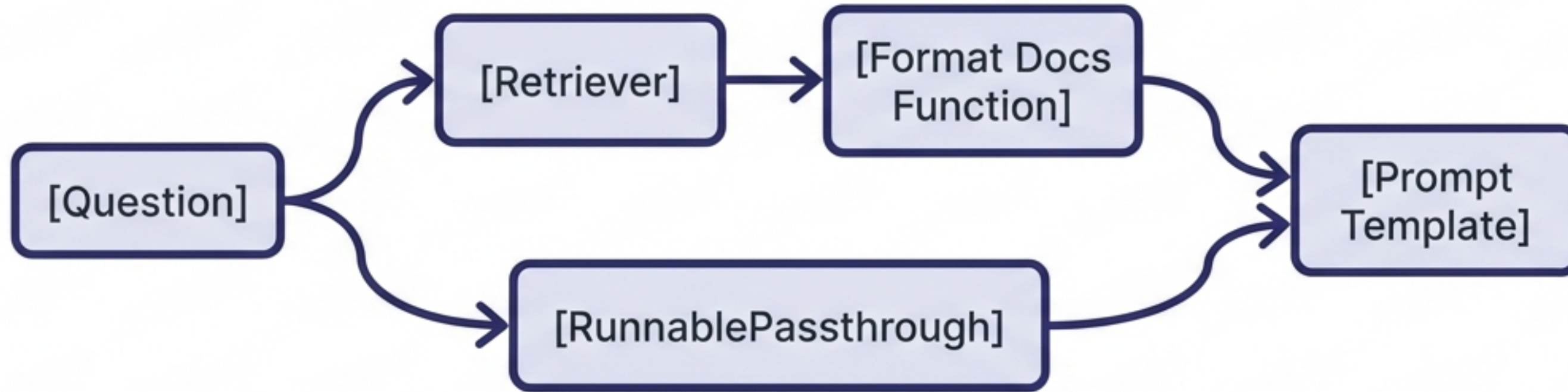
LangChain Expression Language (LCEL)



Declarative composition. Single invocation.

```
chain = RAG | Prompt | LLM | Parser
```


Implementing the RAG Chain



```
...  
from langchain_core.runnables import RunnablePassthrough, RunnableParallel  
  
setup_and_retrieval = RunnableParallel(  
    {"context": retriever | format_docs, "question": RunnablePassthrough()}  
)  
  
rag_chain = setup_and_retrieval | prompt | llm | parser  
  
# Execute  
rag_chain.invoke("Summary of the video?")
```


Moving to Production: UI & Experience

Developer Mode



Google Colab Notebook.
Functional but requires
code execution.

Web App (MVP)



Streamlit App
User pastes URL -> Chat
interface loads. Easy
deployment.

Integrated Product



Chrome Extension
Overlay chat directly on
YouTube. Requires
HTML/JS.

Advanced RAG: Improving Indexing

Optimizing data quality before retrieval (Garbage In, Garbage Out).



Translation: Detect foreign languages (e.g., Hindi) and translate transcripts to English *before* embedding.



Semantic Chunking: Replace fixed character counts with logic that breaks text at natural pauses or topic changes.



Infrastructure: Migrate from local FAISS storage to production-grade Cloud Vector DBs (Pinecone, Weaviate).

Advanced RAG: Smarter Retrieval



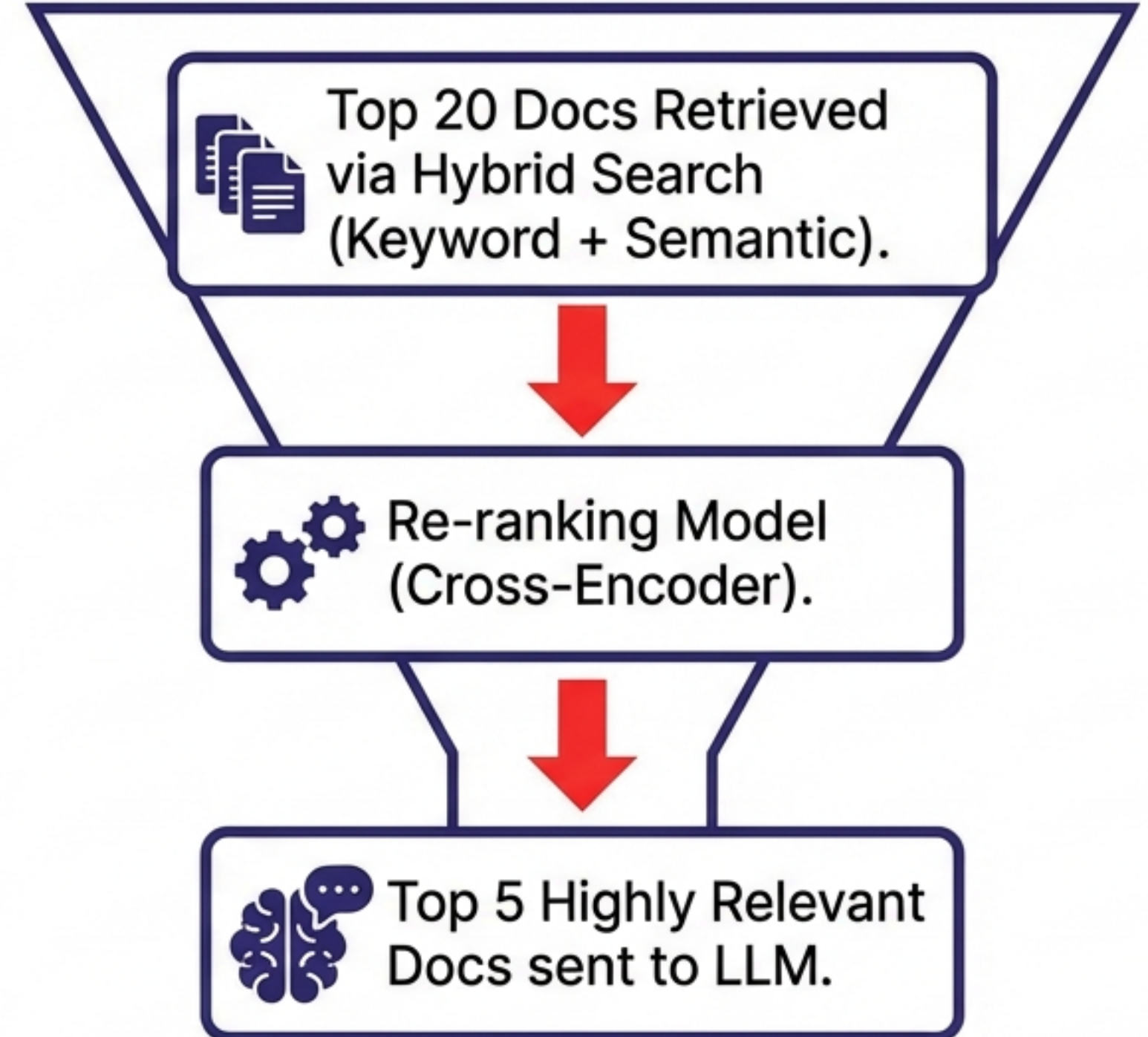
Query Transformation: Rewrite vague queries using an LLM.



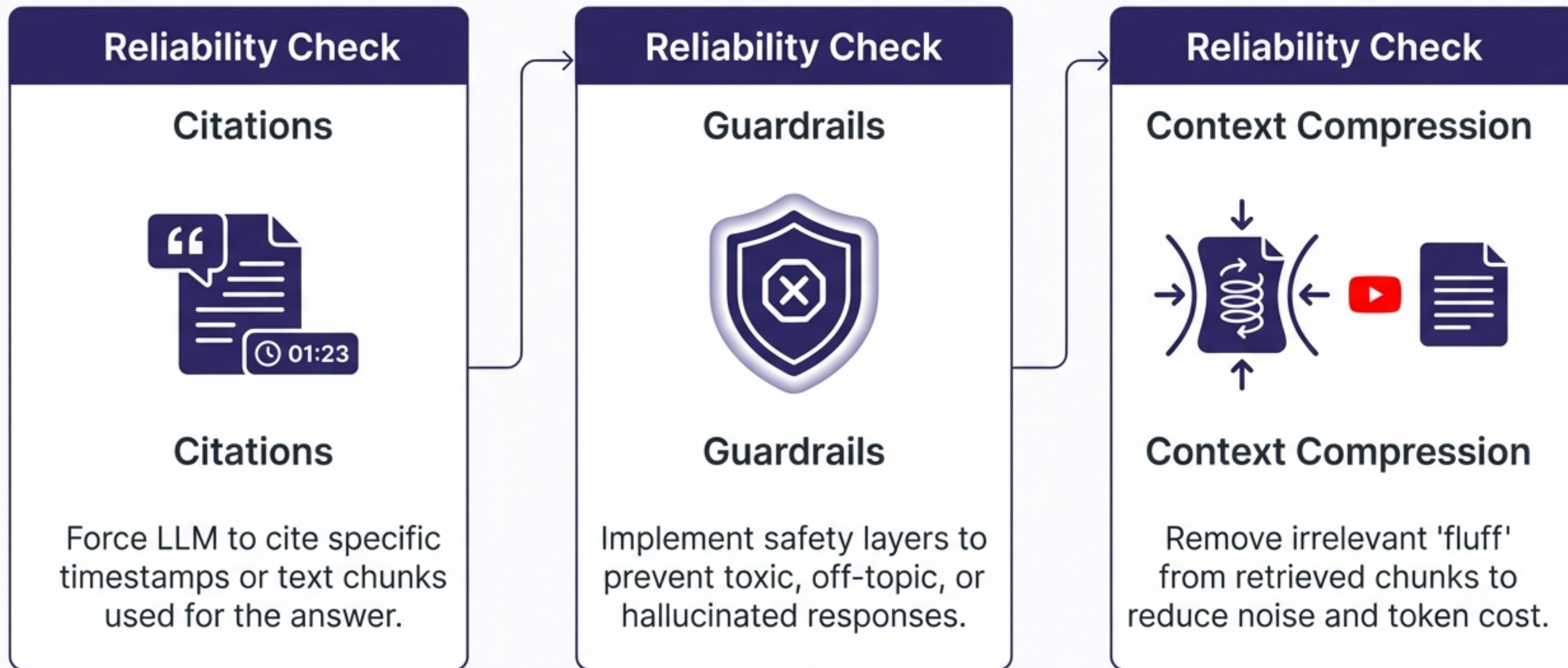
Hybrid Search: Capture specific terminology via keywords + concepts via vectors.



Re-ranking: Re-score retrieval results to filter out false positives.



Advanced RAG: Robust Generation



System Evolution: Beyond Simple Chat



Summary & Next Steps

Recap



- **We Built:** A complete Pipeline (Ingest -> Split -> Embed -> Retrieve -> Generate).



- **We Automated:** Converted manual steps into a clean LCEL Chain.

The Future



- **The Missing Piece:** Evaluation. You cannot improve what you cannot measure.



- **Tooling:** Use Ragas (for faithfulness/relevancy) and LangSmith (for tracing).

This build is the baseline. Real engineering begins with optimization.